S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 05

Aim: Write a program to implement Shortest Job First (SJF) Preemptive Scheduling for three processes and calculate the total context switches and average waiting time. The processes have burst times 10ns, 20ns, and 30ns, arriving at 0ns, 2ns, and 6ns, respectively.

Name: Sonu Shingare

USN: CD24053

Semester / Year: 4th SEM / 2YEAR

Academic Session: 2025-26

Date of Performance:

Date of Submission:

❖ **Aim:** Write a program to implement Shortest Job First (SJF) Preemptive Scheduling for three processes and calculate the total context switches and average waiting time. The processes have burst times 10ns, 20ns, and 30ns, arriving at 0ns, 2ns, and 6ns, respectively.

❖ **Objectives:**

Understand SJF Preemptive Scheduling: Implement the **Shortest Job First (SJF) Preemptive Scheduling** algorithm to manage CPU execution efficiently.

Calculate Context Switches: Determine thetotalnumberofcontext switches required forthe given setofprocesses.

Evaluate Waiting Time: Computethe **average waiting time** forall processes before getting CPU execution.

❖ **Requirements:**

✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

✓ **Software Requirements:**

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

❖ **Theory:**

CPU Scheduling in Operating Systems

Introduction

Scheduling isthe method by which processes are given access to the CPU. Efficient scheduling is essential for optimal system performance and user experience. There are two primary types of CPU scheduling: **Preemptive Scheduling** and **Non-Preemptive Scheduling**.
Understanding the differences between these scheduling types helps in designing and choosing the right scheduling algorithms for different operating systems.

1. Preemptive Scheduling

In **Preemptive Scheduling** ,the operating system can interrupt or preempt a running processtoallocateCPU time to another process, typically based on priority or time-sharingpolicies. A process canbe switched from the **running state to the ready state**atanytime.

Algorithms BasedonPreemptive Scheduling:

- **Round Robin (RR)**
- **Shortest Remaining Time First (SRTF)**
- **Priority Scheduling (Preemptive version)**

Example:

Inthefollowing case, **P2 is preempted at time 1** due to the arrival of a higher-priorityprocess.

Advantages of Preemptive Scheduling:

✓ Prevents a process from monopolizing the CPU, improving system reliability.

✓ Enhances **average response time**, making it beneficial for multi-programming environments.

✓ Used in modern operating systems like **Windows, Linux, and macOS**.

Disadvantages of Preemptive Scheduling:

✗ Morecomplex toimplement.

✗ Involves**overhead** forsuspendingarunningprocessandswitching contexts.

✗ **May cause starvation** iflow-priorityprocessesarefrequently preempted.

✗ Cancreate **concurrency issues**,especiallywhenaccessing shared resources.

2. **Non-Preemptive Scheduling**

In **Non-Preemptive Scheduling** ,arunningprocesscannotbe interrupted by the operatingsystem.Itcontinuesexecutinguntilit **terminates** or **enters a waiting state** voluntarily.

Algorithms Based on Non-Preemptive Scheduling:

- **First Come First Serve (FCFS)**
- **Shortest Job First (SJF - Non-Preemptive)**
- **Priority Scheduling (Non-Preemptive version)**

Example:

Belowis a **Gantt Chart** based onthe **FCFS algorithm**,where each process executes fully beforethenext one starts.

Advantages of Non-Preemptive Scheduling:

✓ **Easy to implement** in an operating system (used in older versions like Windows 3.11 and early macOS).

✓ **Minimal scheduling overhead** due to fewer context switches.

✓ **Lesscomputational resource usage**, making it more efficient for simpler systems.

Disadvantages of Non-Preemptive Scheduling:

✗ **Risk of Denial of Service (DoS) attacks**, as a process can monopolize the CPU.

✗ **Poor response time**, especially in multi-user systems.

3. **Differences Between Preemptive and Non-Preemptive Scheduling**

Parameter	Preemptive Scheduling	Non-Preemptive Scheduling
Basic Concept	CPU time is allocated for a **limited time**.	CPU is held until process **terminates** or enters waiting state.
Interrupts	Process **can be interrupted**.	Process **cannot be interrupted**.
Starvation	Frequent high-priority processes may starve low-priority ones.	A long-running process can starve later-arriving shorter processes.
Overhead	Higher overheaddue to frequent **context switching**.	Minimal overhead.
Flexibility	More flexible(critical processes getpriority).	Rigid schedulingapproach.
Response Time	Faster responsetime.	Slower responsetime.
Process Control	**OS has more control**over scheduling.	OShas **less control** over scheduling.
Concurrency Issues	**Higher**, as processes may be preempted during shared resource access.	**Lower**,as processesrunto completion.
Examples	Round Robin, SRTF.	FCFS, Non-Preemptive SJF.

4. Frequently Asked Questions (FAQs)

a. How is priority determined in Preemptive scheduling?

Ans: Preemptive scheduling systems assign priority based on **task importance, deadlines, or urgency**. Higher-priority tasks execute before lower-priority ones.

b. What happens in non-preemptive scheduling if a process does not yield the CPU?

Ans: If a process does not voluntarily yield the CPU, it can lead to **starvation or deadlock**, where other tasks are unable to execute.

c. Which scheduling method is better for real-time systems?

Ans:Preemptive schedulingisbetterfor **real-time systems**,asitallows high-prioritytasksto executeimmediately .

❖ **CODE:**

```
sonus@DESKTOP-5EMSD6U MINGW64 ~
$ nano SJF.c
```

```
GNU nano 8.7
#include <stdio.h>

int main() {
    int n, i, j;
    int arrival[10], burst[10], temp[10];
    int waiting[10], turnaround[10];
    float total_wt = 0, total_tt = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("Enter arrival time for process %d: ", i+1);
        scanf("%d", &arrival[i]);
        printf("Enter burst time for process %d: ", i+1);
        scanf("%d", &burst[i]);
        temp[i] = burst[i];  // temporary copy of burst times
    }

    // Sorting processes by arrival time
    for(i = 0; i < n-1; i++) {
        for(j = i+1; j < n; j++) {
            if(arrival[i] > arrival[j]) {
                int t = arrival[i];
                arrival[i] = arrival[j];
                arrival[j] = t;

                t = burst[i];
                burst[i] = burst[j];
                burst[j] = t;

                t = temp[i];
                temp[i] = temp[j];
                temp[j] = t;
            }
        }
    }

    int time = 0;
    int completed = 0;
    int is_done[10] = {0};

    while(completed < n) {
        int idx = -1;
        int min_bt = 9999;

        for(i = 0; i < n; i++) {
            if(arrival[i] <= time && !is_done[i] && burst[i] < min_bt) {
                min_bt = burst[i];
```

```
Help        Write Out   Where Is    Cut         Execute
Exit        Read File   Replace     Paste       Justify
```

```
                idx = i;
            }
        }

        if(idx != -1) {
            waiting[idx] = time - arrival[idx];
            time += burst[idx];
            turnaround[idx] = waiting[idx] + temp[idx];
            is_done[idx] = 1;
            completed++;
        } else {
            time++; // increment time if no process has arrived yet
        }
    }

    printf("\nProcess\tArrival\tBurst\tWaiting\tTurnaround\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\n", i+1, arrival[i], temp[i], waiting[i], turnaround[i]);
        total_wt += waiting[i];
        total_tt += turnaround[i];
    }

    printf("\nAverage Waiting Time: %.2f\n", total_wt/n);
    printf("Average Turnaround Time: %.2f\n", total_tt/n);

    return 0;
}
```

```
Help        Write Out   Where Is    Cut         Execute     Locatio
Exit        Read File   Replace     Paste       Justify     Go To L
```

❖ Output:

```
sonus@DESKTOP-5EMSD6U MINGW64 ~
$ gcc SJF.c -o SJF && ./SJF
Enter number of processes: 2
Enter arrival time for process 1: 4
Enter burst time for process 1: 2
Enter arrival time for process 2: 3
Enter burst time for process 2: 1

Process Arrival Burst   Waiting Turnaround
P1      3       1       0       1
P2      4       2       0       2

Average Waiting Time: 0.00
Average Turnaround Time: 1.50
```

Conclusion: Preemptive scheduling offers better responsiveness but adds complexity, while non-preemptive scheduling is simpler but may cause inefficiencies. The choice depends on system needs, with preemptive suited for multitasking and non-preemptive for low-overhead scenarios.

❖ Discussion Questions:

1. **What is the key difference between preemptive and non-preemptive scheduling?**
   Ans: Preemptive scheduling allows the CPU to betaken away from a process beforeit finishes execution, whereas non-preemptivescheduling ensures a process runs until completion or voluntary release of the CPU.
2. **Why does preemptive scheduling require context switching?**
   Ans: Preemptive scheduling interrupts running processes, requiring the system to save the current state (context) and load the next process, leading to context switchingoverhead.
1. **Which CPU scheduling algorithm is most suitable for real-time systems and why?**
   Ans: Preemptive priority scheduling
   is best forreal-timesystemsasit ensures high-priority tasksgetimmediate CPU access,reducingresponse time for critical tasks.

2. What is starvation in CPU scheduling, and how can it be prevented? Ans: Starvation occurs when low-priority processes wait indefinitely due to frequent arrival of high-priority tasks. It can be prevented using **aging**, which gradually increases a process's priority over time.

3. Why is the Round Robin scheduling algorithm preferred in time-sharing systems?

Ans: Round Robin ensures **fair CPU allocation** byassigningtimeslices (quantum) to each process, preventingmonopolizationandprovidinga balanced response time in multi-user environments.

❖ **References:**

https://www.geeksforgeeks.org/preemptive-and-non-preemptive-scheduling/

Date:____/____/2026

: 4 / 2025-26